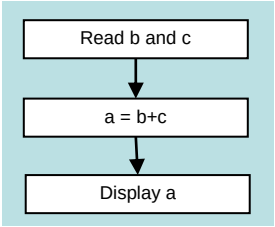
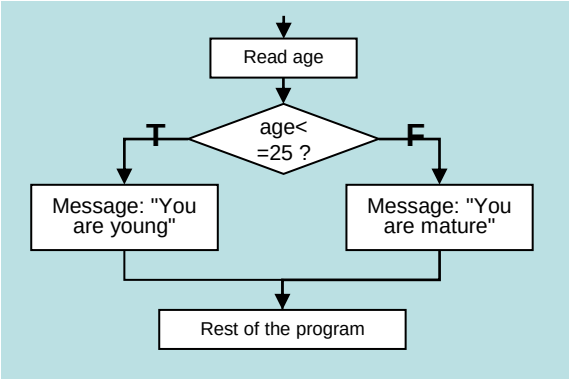
# Control of flow

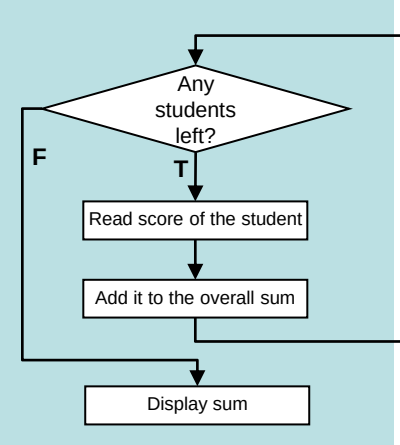
- We learned that default flow of instructions is sequential.




- Then, we learned how to control the flow using "if" and "switch."




- Now, we will learn how to *repeat* a group of instructions.



# Types of loops

- There are three types of loops:
  - "for" loop
  - "while" loop
  - "do-while" loop
- You can implement anyone of these loops using the others (and possibly an additional if statement and duplication of some statements).
  - The idea is to use the type of loop that is best suited for your problem.

# General structure of a loop

- A loop is typically composed of three parts:
  - the statement block which is to be repeated;
  - a control mechanism to decide whether the statement block should be repeated once more (based on an expression);
  - an update mechanism that affects the value of the control expression (to avoid infinite loops).
- The update mechanism may be embedded in the statement block.

# General structure of a loop

- One execution of the statement block is called an iteration.
- Thus, a loop *iterates* the statement block several times.
- Make sure:
  - it is possible to enter the loop;
  - it is possible to get out of the loop, once you enter it.

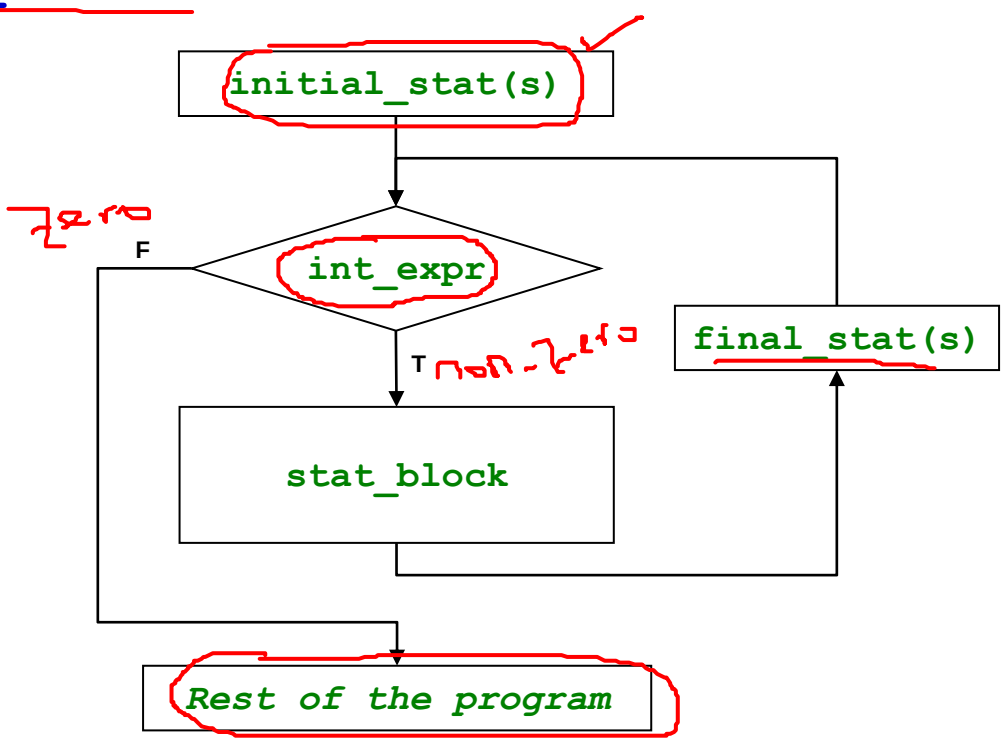
# For loop

- Use for loop if you know the number of iterations.
  - You don't have to know the exact number of iterations; it is enough if you can express it.

# For loop

- Syntax:

```
for (initial_stat(s);int_expr; final_stat(s))
    stat_block
```



# For loop

- Note that initial and final statements as well as the integer expression are optional according to the syntax.
  - If **`initial_stat(s)`** is missing, start directly with the comparison.
  - If **`final_stat(s)`** is missing, go directly to the comparison after the execution of the statement block.
  - If **`int_expr`** is missing, the comparison is always true.
    - Make use of the **`break`** statement (to be discussed later).

# for: Example 1

Write a code segment that prints the text "I didn't do my homework" 100 times with every line enumerated.

```
printf("1.  I didn't do my homework.\n");
printf("2.  I didn't do my homework.\n");
printf("3.  I didn't do my homework.\n");
printf("4.  I didn't do my homework.\n");
printf("5.  I didn't do my homework.\n");
printf("6.  I didn't do my homework.\n");
printf("7.  I didn't do my homework.\n");
printf("8.  I didn't do my homework.\n");
printf("9.  I didn't do my homework.\n");
printf("10. I didn't do my homework.\n");
printf("11. I didn't do my homework.\n");
printf("12. I didn't do my homework.\n");
printf("13. I didn't do my homework.\n");
printf("14. I didn't do my homework.\n");
printf("15. I didn't do my homework.\n");
printf("16. I didn't do my homework.\n");
printf("17. I didn't do my homework.\n");
printf("18. I didn't do my homework.\n");
printf("19. I didn't do my homework.\n");
printf("20. I didn't do my homework.\n");
.
.
.
printf("91. I didn't do my homework.\n");
printf("92. I didn't do my homework.\n");
printf("93. I didn't do my homework.\n");
printf("94. I didn't do my homework.\n");
printf("95. I didn't do my homework.\n");
printf("96. I didn't do my homework.\n");
printf("97. I didn't do my homework.\n");
printf("98. I didn't do my homework.\n");
printf("99. I didn't do my homework.\n");
printf("100.I didn't do my homework.\n");
```

# for: Example 1 *(cont'd)*

Lazy student's solution ☺

```
int i;

   for (i=0; i<100; i++)
      printf("%d. I didn't do my homework.\n", i);
```

# for: Example 2

Find $a^b$. (a and b integers)

```
int a, b, result=1, i;

scanf("%d %d", &a, &b);
for (i=0; i<b; i++)
    result *= a;
```

# for: Example 2 *(cont'd)*

Same question, solved with fewer variable (but the value of **b** changes).

```
int a, b, result=1;

scanf("%d %d", &a, &b);
for (; b; b--)
    result *= a;
printf("result=%d", result);
```

# for: Example 3

Find the average of the midterm scores of the students in CSE1145.

```
int i, sum, no_stu, score;
float avg;

scanf("%d", &no_stu);
for (sum=i=0; i<no_stu; i++)
{
    scanf("%d", &score);
    sum += score;
}
avg = sum/no_stu;
```

What is wrong here?

What else?

What if "no_stu" is zero?

# for: Example 4

Calculate BMI (Body Mass Index) of all students in class ☹

- weight/height

```
for (i=0; i<100; i++)
{
    scanf("%d %d", &weight, &height);
    printf("BMI: %f\n", (float)weight/height);
}
```

What if "height" is zero?

# for: Example 5

What are the differences between these code segments?

| | | |
|---|---|---|
| `for (i=0; i<5; i++)`<br>`  printf("%d;", i);`<br>`printf("{%d}",i);` | `for (i=0; i<=5; i++)`<br>`  printf("%d;", i);`<br>`printf("{%d}",i);` | `for (i=1; i<5; i++)`<br>`  printf("%d;", i);`<br>`printf("{%d}",i);` |
| `0;1;2;3;4;{5}` | `0;1;2;3;4;5;{6}` | `1;2;3;4;{5}` |
| `for (i=1; i<=5; ++i)`<br>`  printf("%d;", i);`<br>`printf("{%d}",i);` | `for (■; i<5; i++)`<br>`  printf("%d;", i);`<br>`printf("{%d}",i);` | `for (i=0;■; i++)`<br>`  printf("%d;", i);`<br>`printf("{%d}",i);` |
| `1;2;3;4;5;{6}` | `Starts from anything ...;3;4;{5}`<br>`OR MAYBE SOMETHING LIKE {795}` | `0;1;2;...∞` |
| `for (i=0; i<5;■)`<br>`  printf("%d;", i);`<br>`printf("{%d}",i);` | `for (i=0; i<5;■)`<br>`  printf("%d;",i++);`<br>`printf("{%d}",i);` | `for (i=0; i++<5;■)`<br>`  printf("%d;", i);`<br>`printf("{%d}",i);` |
| `0;0;0;0;...∞` | `0;1;2;3;4;{5}` | `1;2;3;4;5;{6}` |

# for: Example 6

Now, compare these code segments.

| | | |
|---|---|---|
| `for (i=7; i<5; i++)`<br>`  printf("%d;", i);`<br>`printf("{%d}",i);` | `for (i=7;++i<5;■)`<br>`  printf("%d;", i);`<br>`printf("{%d}",i);` | `for (i=7;i++<5;■)`<br>`  printf("%d;", i);`<br>`printf("{%d}",i);` |
| `{7}` | `{8}` | `{8}` |

| | |
|---|---|
| `for (i=7; ++i<5; ++i)`<br>`  printf("%d;",++i);`<br>`printf("{%d}",i);` | `for (i=0; i<5; ++i)`<br>`  printf("%d;",++i);`<br>`printf("{%d}",i);` |
| `{8}` | `1;3;5;{6}` |

# for: Example 7

- What if Carl Friedrich Gauss knew how to program when he was in elementary school?
  - Let's be more generic; add numbers from 1 to n.

```
scanf("%d", &n);
for (sum=0, i=1; i<=n; i++)
    sum += i;
```

- Of course Gauss would be clever enough to do

```
scanf("%d", &n);
sum = n * ((n+1)/2);
```

  - What he right?

# for: Example 8

Find whether a number is perfect or not.

(A number is perfect if sum of its positive divisors except itself is equal to itself. Eg: 6=1+2+3; 28=1+2+4+7+14)

```
int number, i, sum = 0;

scanf("%d",&number);
for (i = 1; i <= number / 2 ; i++)
    if (number % i == 0)
        sum += i;
if (number == sum)
    printf("%d is a perfect number",number);
else
    printf("%d is not a perfect number",number);
```